

2025-2026

Expert en informatique et système d'information

BLOC 2 – Manager un projet informatique avec agilité en
collaboration avec les parties prenantes

MSPR « Gérer un projet d'Infrastructure virtualisée Agile et
multiculturel »

Groupe :

MAZOUZ Mehdi

YILMAZ Ammar

NJIME-AKWA Moudio

MOBEKO Edouard Junior

HOUNKANRIN Jean-Gérard

Remerciements

On tient tout d'abord à remercier Mme DAUCA Anne-Sophie et à l'ensemble du corps professionnel de l'EPSI Paris pour l'ensemble de leurs œuvres et enseignements qui nous ont permis d'arriver au bout de notre objectif durant ce projet.

Nos remerciements s'adressent également à l'ensemble de notre équipe qui ont pris de leur temps et de leur expérience pour accomplir ce projet.

Nos derniers remerciements s'adressent à toutes les personnes qui ont été disponibles pour nous ainsi qu'à nos camarades de classe et tous ceux qui nous ont aidés à évoluer en compétences.

Table des matières

Remerciements	2
Table des matières	3
Introduction	4
Missions	6
M1. Benchmark et choix des technologies	6
M2 : Organisation du projet.....	9
M3 : Suivi de l'avancement du projet.....	15
M4 : Packer.....	18
M5 : Terraform	20
M6 : Ansible, déploiement de Kubernetes	21
M7 : Ansible, déploiement d'Odoo et mise en place de l'Ingress.....	22
Conclusion.....	24
Annexes.....	25
Annexe 1 : Diagramme de Gantt.....	25
Annexe 2 : Environnement de travail inclusif, tolérant et multiculturel	26
Annexe 3 : Scripts Packer.....	28
Annexe 4 : Scripts Packer.....	30
Annexe 5 : Scripts Ansible pour le déploiement de Kubernetes	33
Annexe 6 : Scripts Ansible pour le déploiement d'Odoo.....	37

Introduction

La société COGIP est un acteur majeur dans le développement de logiciels de gestion d'entreprise, offrant des solutions adaptées aux besoins des PME et des grandes entreprises. Avec des produits phares comme l'ERP libre Odoo et leur solution propriétaire CogiCloud, COGIP accompagne ses clients dans la gestion globale de leurs activités.

Récemment, COGIP a remporté un contrat stratégique avec Tesker, un fabricant de véhicules électriques en pleine expansion, nécessitant une infrastructure informatique robuste, flexible et résiliente. Face à cet enjeu, notre mission est de concevoir et de déployer une infrastructure basée sur Kubernetes, capable de supporter les applicatifs de COGIP, tout en garantissant évolutivité et continuité des services grâce à des outils d'Infrastructure as Code (IaC).

Ce projet s'inscrit dans une démarche Agile pour optimiser la collaboration et les délais. Il inclut la mise en place d'un cluster Kubernetes, le déploiement de l'ERP Odoo et la livraison d'un Proof of Concept (PoC) démontrant l'efficacité et la reproductibilité de la solution. Cette initiative illustre l'engagement de COGIP à répondre aux exigences technologiques de ses clients tout en restant à la pointe de l'innovation.

Voici la répartition des tâches de notre équipe :

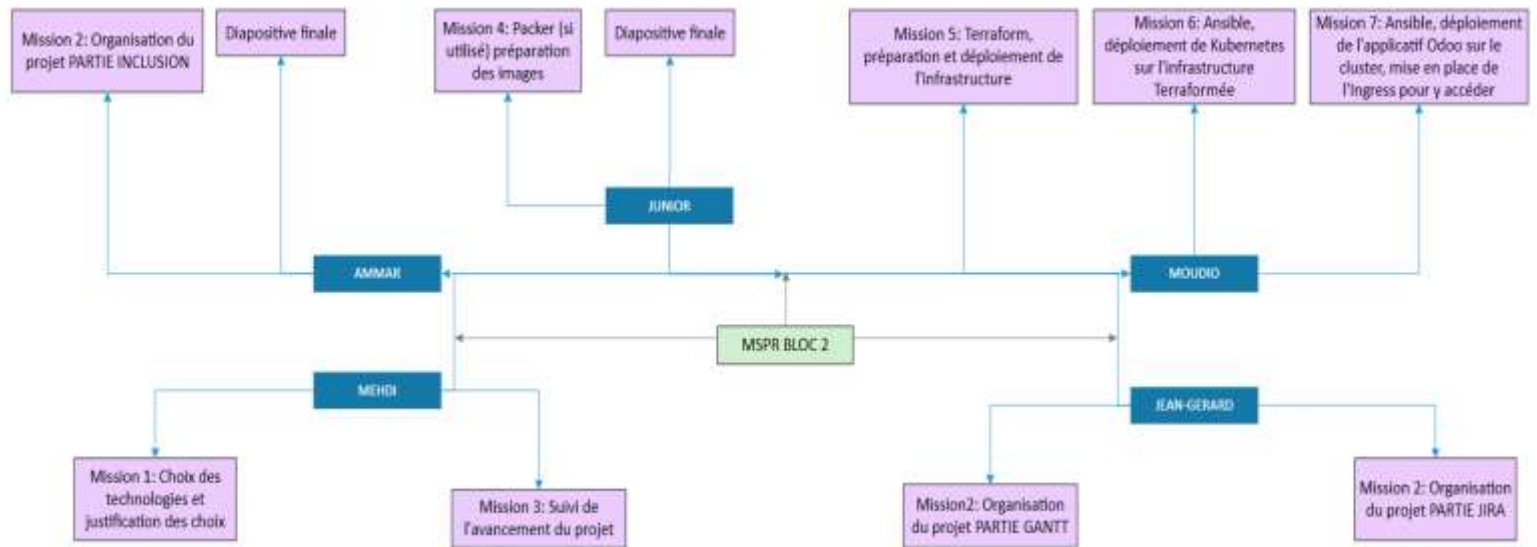


Figure 1 : Répartition des tâches

Missions

M1. Benchmark et choix des technologies

Dans le cadre du projet pour la société COGIP et son client Tesker, nous avons mené une veille technologique approfondie. L'objectif est de proposer une infrastructure évolutive, résiliente et hautement automatisable (Infrastructure as Code) permettant un Plan de Reprise d'Activité (PRA) rapide et fiable.

Nos choix se sont portés sur trois axes majeurs : la distribution Kubernetes, le type d'hébergement, et les outils d'automatisation.

1. Benchmark des distributions Kubernetes

Notre critère principal pour ce Proof of Concept (PoC) était de trouver un équilibre entre légèreté, rapidité de mise en œuvre, et intégration native des outils réseaux (Ingress/LoadBalancer) pour héberger l'ERP Odoo.

Technologie	Solution étudiée	Avantages clés	Inconvénients / Limites	Décision & Justification
Kubernetes	Suse K3s	Ultra-léger, déploiement très rapide. Intègre nativement un LoadBalancer (svcLB) et un Ingress (Traefik).	Moins adapté par défaut pour des clusters gigantesques (entreprise mondiale).	RETENU : Parfait pour notre PoC. L'intégration de Traefik et svcLB nous fait gagner un temps précieux tout en garantissant évolutivité et reproductibilité.
Kubernetes	Suse RKE2	Haut niveau de sécurité (orienté gouvernement/banques).	Nécessite le déploiement manuel d'un Ingress (ex: ingress-nginx).	ÉCARTÉ : Trop lourd et complexe pour notre besoin actuel. Les réglages manuels ralentiraient la phase de PoC.
Kubernetes	Canonical MicroK8s	Excellente intégration à l'écosystème Ubuntu.	Activation des composants (add-ons) entièrement manuelle. Parfois instable hors d'Ubuntu.	ÉCARTÉ : Manque d'agnosticité au niveau de l'OS et demande de nombreux ajustements manuels.
Kubernetes	Mirantis K0s	Solution 100% "vanilla", très pure et sans aucune dépendance.	Aucun outil réseau fourni. Obligation d'installer soi-même MetalLB et Ingress-Nginx.	ÉCARTÉ : Risque de perte de temps sur la configuration réseau bas niveau, au détriment du déploiement de l'applicatif.

Dans le cadre de la mission, nous avons mené une veille technologique afin de choisir la distribution Kubernetes la plus adaptée à notre projet. Les solutions étudiées sont : Suse K3S, Suse RKE2, Mirantis K0S et Canonical MicroK8S. Nos critères de sélection portaient sur la simplicité d'installation, la maintenance, la légèreté, la compatibilité avec des environnements à ressources limitées, ainsi que la rapidité de mise en œuvre, notamment dans le cadre d'un Proof of Concept (PoC).

Parmi ces solutions, Suse K3S s'est démarquée par sa facilité de déploiement, sa faible empreinte

système et l'intégration native de composants essentiels comme le LoadBalancer (svcLB) et l'Ingress (Traefik). Ces avantages permettent un déploiement rapide et reproductible sur un cluster composé d'un control-plane et de deux workers, tout en garantissant une bonne évolutivité.

À l'inverse, Suse RKE2 et Mirantis K0S exigent une configuration plus manuelle (notamment pour le LoadBalancer et l'Ingress), ce qui peut complexifier le déploiement, bien qu'ils offrent davantage de personnalisation pour des environnements plus exigeants. Canonical MicroK8S se montre également simple à installer et dispose d'add-ons pratiques, mais peut nécessiter des ajustements pour atteindre une configuration de production stable.

En conclusion, au regard de nos objectifs et de notre contexte technique, nous avons retenu Suse K3S comme la distribution Kubernetes la plus adaptée pour notre projet.

2. Choix de la solution d'hébergement

Pour héberger nos machines virtuelles (1 control-plane, 2 workers), nous devons trancher entre une solution Cloud Managée et une infrastructure "Baremetal" (Hyperviseur local).

Solution étudiée	Type	Avantages	Inconvénients	Décision / justification
Hyperviseur local Ex. : Proxmox / VMware	Baremetal / On-Premise	Gratuité pour le PoC. Maîtrise complète de la chaîne technique, de la création des VM jusqu'au déploiement applicatif.	Configuration réseau et stockage à gérer manuellement. Mise en place du stockage persistant, par exemple via NFS.	RETENU Ce choix permet de démontrer notre maîtrise complète de l'automatisation, sans dépendre d'un fournisseur Cloud.
Google Kubernetes Engine GKE	Cloud public managé	Forte scalabilité. Gestion automatisée du control-plane, du LoadBalancer et du stockage par Google.	Coût financier récurrent. Une partie de la complexité technique est masquée par le fournisseur Cloud.	ECARTE. Solution adaptée à une production finale, mais moins pertinente pour ce PoC car elle limite la démonstration du déploiement d'un cluster from scratch.

3. Justification des outils d'Infrastructure as Code (IaC)

La COGIP a formellement exigé une solution facilement reproductible afin de garantir un Plan de Reprise d'Activité (PRA) efficace. Pour répondre à cette contrainte de résilience, nous avons opté pour une approche 100% Code centralisée sur un dépôt Git, divisée en trois briques complémentaires :

Outil retenu	Rôle dans notre architecture	Justification par rapport au besoin COGIP (PRA)
Packer	Création des images "Golden" (Baking)	Permet de créer des images systèmes pré-configurées (OS, clés SSH, paquets de base). En cas de crash, nous ne repartons pas de zéro, mais d'une image standardisée, propre et sécurisée.
Terraform	Provisionnement de l'infrastructure (Build)	Agit comme le chef de chantier. Il lit nos recettes et déploie les VMs (réseau, ressources CPU/RAM) de manière identique à chaque lancement. En cas de désastre majeur, un simple terraform apply recrée le datacenter virtuel exact.
Ansible	Configuration et Déploiement (Configure)	Outil idempotent qui va installer K3s sur nos VMs, puis déployer Odoo (via le module Helm). Ansible garantit que l'état final des serveurs correspond exactement à ce qui est écrit dans le code, évitant ainsi les erreurs humaines.

L'association de Packer, Terraform et Ansible combinée à Suse K3s nous permet de livrer à la COGIP un PoC robuste. Si le datacenter de Tesker venait à subir une panne totale, notre équipe serait capable de redéployer l'intégralité de l'infrastructure et de l'applicatif Odoo en quelques minutes seulement, en exécutant simplement nos scripts versionnés.

M2 : Organisation du projet

Notre projet Scrum débute par un Product Backlog, une liste priorisée de besoins (fonctionnalités, corrections, demandes métiers) formulés en User Stories avec des critères d'acceptation clairs et une estimation de charge. Lors de la Sprint Planning, nous avons défini un objectif et nous avons sélectionner les User Stories à réaliser selon sa capacité. Ces éléments une fois découpés en tâches techniques, forment le Sprint Backlog, réparti de manière collaborative en tenant compte des compétences et contraintes de chacun. Voici notre backlog :

Epic 1 – Mise en place de l'environnement de virtualisation

US 1.1 – En tant qu'admin infra, je veux disposer des ISO et images à jour

- Tâche : lister les ISO requis (ESXi, vCenter, Ubuntu...)
- Tâche : télécharger et valider sommes de contrôle (checksums)

US 1.2 – En tant qu'admin infra, je veux installer et configurer ESXi

- Tâche : installation ESXi sur le poste local
- Tâche : configuration réseau de management, datastore
- Tâche : tests création VM manuelle

US 1.3 – En tant qu'admin infra, je veux installer et configurer vCenter

- Tâche : déployer Appliance vCenter sur ESXi
- Tâche : configurer domaine (SSO), créer datacenter
- Tâche : vérifier accès via vSphere Client

Epic 2 – Création du Template Ubuntu & outillage IaC

US 2.1 – En tant qu'admin infra, je veux un template Ubuntu standardisé

- Tâche : déployer VM Ubuntu depuis ISO
- Tâche : configurer SSH, utilisateurs, hostname, firewall
- Tâche : installer dépendances (python, openssh-server...)
- Tâche : convertir VM en Template

US 2.2 – En tant qu'admin infra, je veux disposer de Terraform, Ansible et Packer

- Tâche : installer Terraform
- Tâche : installer Ansible
- Tâche : rédiger le Template Packer (Builder vSphere, variables ISO/user/clé SSH/paquets)

- Tâche : valider init Terraform & commande ansible --version

Epic 3 – Provisionnement des VMs Kubernetes avec Terraform

US 3.1 – En tant qu’admin infra, je veux un code Terraform pour créer les VMs

- Tâche : écrire main.tf (provider vSphere, data template, resource VM)
- Tâche : définir variables (nombre de CPU, RAM, datastores...)
- Tâche : tester terraform plan/apply et terraform destroy

Epic 4 – Installation et configuration du cluster Kubernetes & déploiement Odoo

US 4.1 – En tant qu’admin infra, je veux un playbook Ansible pour installer k3s

- Tâche : inventaire Ansible (control-plane + 2 workers)
- Tâche : playbook install-k3s.yaml (install k3s server, récup token, join agents)
- Tâche : playbook config réseau (flannel/traefik auto)
- Tâche : valider kubectl get nodes

US 4.2 – En tant qu’opérateur, je veux déployer Odoo sur ce cluster

- Tâche : playbook helm-repos.yaml (ajout repo bitnami)
- Tâche : playbook deploy-odoo.yaml (Chart Odoo, PVC, service, ingress)
- Tâche : playbook config cert auto-signé via cert-manager
- Tâche : valider accès <https://odoo.local>

Sprint 1 : Mettre en place l’environnement de virtualisation (ESXi + vCenter)

Durée = 2 semaines Stories

sélectionnées :

US 1.1 – Récupérer et valider les ISO US

1.2 – Installer et configurer ESXi US 1.3 –

Déployer et configurer vCenter

Tâches Sprint 1 :

- Lister les ISO nécessaires (ESXi, vCenter, Ubuntu)
- Télécharger les ISO et vérifier leurs checksums
- Installer ESXi sur le poste local
- Configurer le réseau de management et le datastore ESXi
- Créer manuellement une VM de test pour valider l’installation d’ESXi

- Déployer l'Appliance vCenter sur ESXi
- Configurer SSO, créer datacenter dans vCenter
- Vérifier l'accès et les droits via le vSphere Client

Sprint 2 : Produire un template Ubuntu et préparer l'outillage IaC + début du code

Terraform

Durée = 2 semaines Stories

sélectionnées :

US 2.1 – Créer un template Ubuntu

US 2.2 – Installer Terraform & Ansible US

3.1 – Démarrer le code Terraform

Tâches Sprint 2 :

- Déployer une VM Ubuntu depuis ISO
- Configurer SSH, comptes utilisateurs, hostname, firewall
- Installer Python, openssh-server...
- Convertir la VM en template dans vCenter
- Installer Terraform sur la station admin
- Installer Ansible sur la station admin
- Rédiger le template Packer
- Vérifier “terraform --version” et “ansible --version”
- Écrire le fichier main.tf (provider vSphere, data template, ressources VM)
- Définir variables.tf (CPU, RAM, datastore, nombre de VMs)
- Lancer un “terraform init” et “terraform plan”

Sprint 3 : Provisionner les VMs, déployer le cluster k3s et installer Odoo

Durée = 2 semaines Stories

sélectionnées :

US 4.1 – Déployer un cluster k3s avec Ansible US

4.2 – Installer Odoo + Ingress via Ansible

Tâches Sprint 3 :

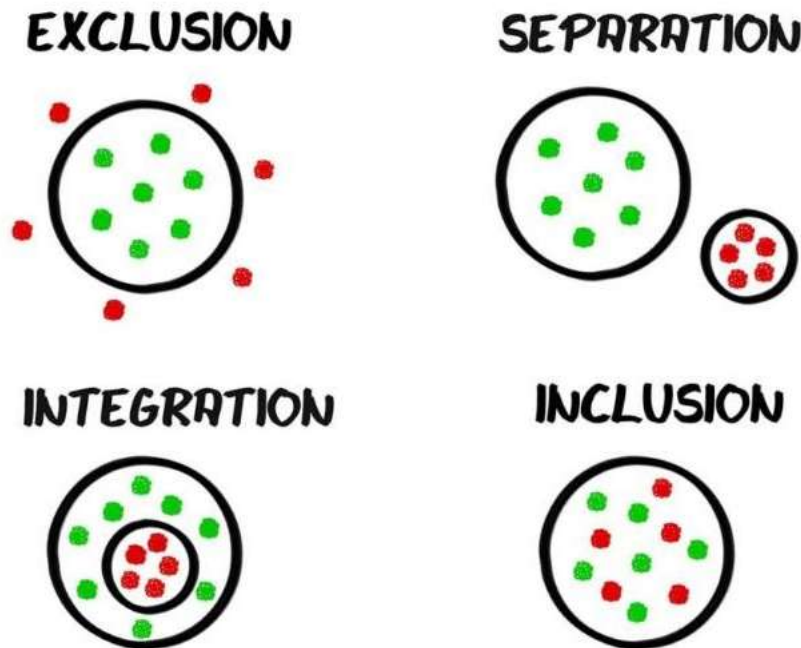
- Exécuter “terraform apply” pour créer control-plane + 2 workers
- Valider la création des VMs dans vCenter
- Rédiger l'inventaire Ansible (groupes control-plane / workers)
- Écrire playbook install-k3s.yaml (installation k3s server, récupération du token, join

agents)

- Écrire playbook config-net.yaml (traefik/Flannel auto-provisionnés par k3s)
- Vérifier “kubectrl get nodes” et “kubectrl get pods -A”
- Écrire playbook helm-repos.yaml (ajout du repo bitnami)
- Écrire playbook deploy-odoo.yaml (Chart Odoo, PVC, service, ingress)
- Écrire playbook cert-manager.yaml (déploiement cert-manager pour TLS auto-signé)
- Tester l'accès à <https://odoo.local> et vérifier la création du pod Odoo
- Nettoyer et documenter les commandes de rollback (ansible-playbook & terraform destroy)

Nous avons mis en place le diagramme de Gantt (**annexe 1**) et nous avons également étudié différentes pistes de réflexion concernant la mise en place d'un environnement de travail multiculturel et inclusif pour permettre aux équipes de favoriser la collaboration, le bien-être et la performance (**annexe 2**).

Philosophie et vision de l'inclusion



Pour notre équipe, la réussite de ce projet dépasse le simple cadre technique. Nous avons structuré notre organisation autour d'un environnement de travail inclusif, garantissant le respect des spécificités de chacun. L'objectif est de lever les barrières physiques, personnelles ou culturelles afin de permettre à chaque membre de s'investir pleinement et de libérer son potentiel sans contrainte limitative.

Accessibilité et aménagement des situations de handicap

L'inclusion opérationnelle nécessite une anticipation rigoureuse. En collaboration avec le référent handicap de l'entreprise, nous avons formalisé des réponses adaptées aux différents profils pour garantir une réelle équité d'accès aux ressources du projet :

- Handicap moteur : Nous aménageons des rythmes avec un recours facilité au télétravail, complété par l'attribution d'un matériel ergonomique adapté à domicile.
- Handicap visuel : Tous nos documents et outils intègrent des descriptions textuelles pour être facilement lisibles par les synthèses vocales.
- Contrainte médicale : Nous lisons la charge de travail et restons très flexibles sur les délais internes pour respecter la santé de chacun sans impacter la roadmap globale.

Management interculturel et communication

Travailler dans un environnement multiculturel est une force, à condition de bien communiquer en adoptant les bons modes selon les cultures. Pour anticiper et désamorcer les conflits liés aux malentendus multiculturels, nous avons rédigé une charte de communication bienveillante. En complément, nous proposons des solutions innovantes comme des sessions d'ice-breaker régulières pour créer du lien humain au-delà de la technique.

Une organisation juste et collaborative

Pour que tout le monde se sente acteur du projet, la transparence est clé. Notre tableau Kanban permet à toute l'équipe de suivre l'avancement de façon équitable avec une visibilité complète sur l'état d'avancement du projet. Nous répartissons les tâches selon les forces de chacun et nous misons beaucoup sur le travail en binôme (peer programming / peer working). Les revues techniques sont systématiquement faites collectivement, ce qui évite l'isolement et tire notre travail vers le haut.

Équilibre de vie et management de la confiance

La performance à long terme est directement corrélée à la qualité de vie au travail. Nous avons structuré notre pratique du télétravail autour d'un management basé sur la confiance mutuelle et l'autonomie. Le respect strict du droit à la déconnexion et des horaires de chacun est une règle d'équipe, garantissant une étanchéité saine entre vie professionnelle et vie privée.

Outils collaboratifs et environnement technique

La distance ne doit pas nous éloigner. Nos outils numériques sont configurés pour mettre en place un fil de discussion régulier et synchroniser facilement nos activités.

- Teams / Slack : Permet de se synchroniser au quotidien et d'animer des réunions à distance (Daily Stand-up) pour maintenir une forte dynamique de groupe.
- Git : Facilite le travail technique en asynchrone sans jamais bloquer les autres membres.
- Kanban : Offre une vision claire et rassurante sur le travail restant à accomplir.

En fin de compte, cette démarche d'inclusion n'est pas une simple consigne : c'est le moteur qui a soudé notre équipe, fluidifié notre communication et boosté notre performance technique, tout en restant fidèles à nos valeurs humaines.

M3 : Suivi de l'avancement du projet

Pour assurer un suivi efficace et structuré de l'avancement du projet pour la COGIP, nous avons utilisé JIRA comme outil central de gestion agile. En nous basant sur le Product Backlog défini lors de la Mission 2, nous avons pu répartir nos Epics et nos User Stories au sein de différents Sprints afin de garantir des livraisons régulières et itératives. Chaque tâche a été chiffrée à l'aide de Story Points (suite de Fibonacci) pour estimer précisément la charge de travail (vélocité).

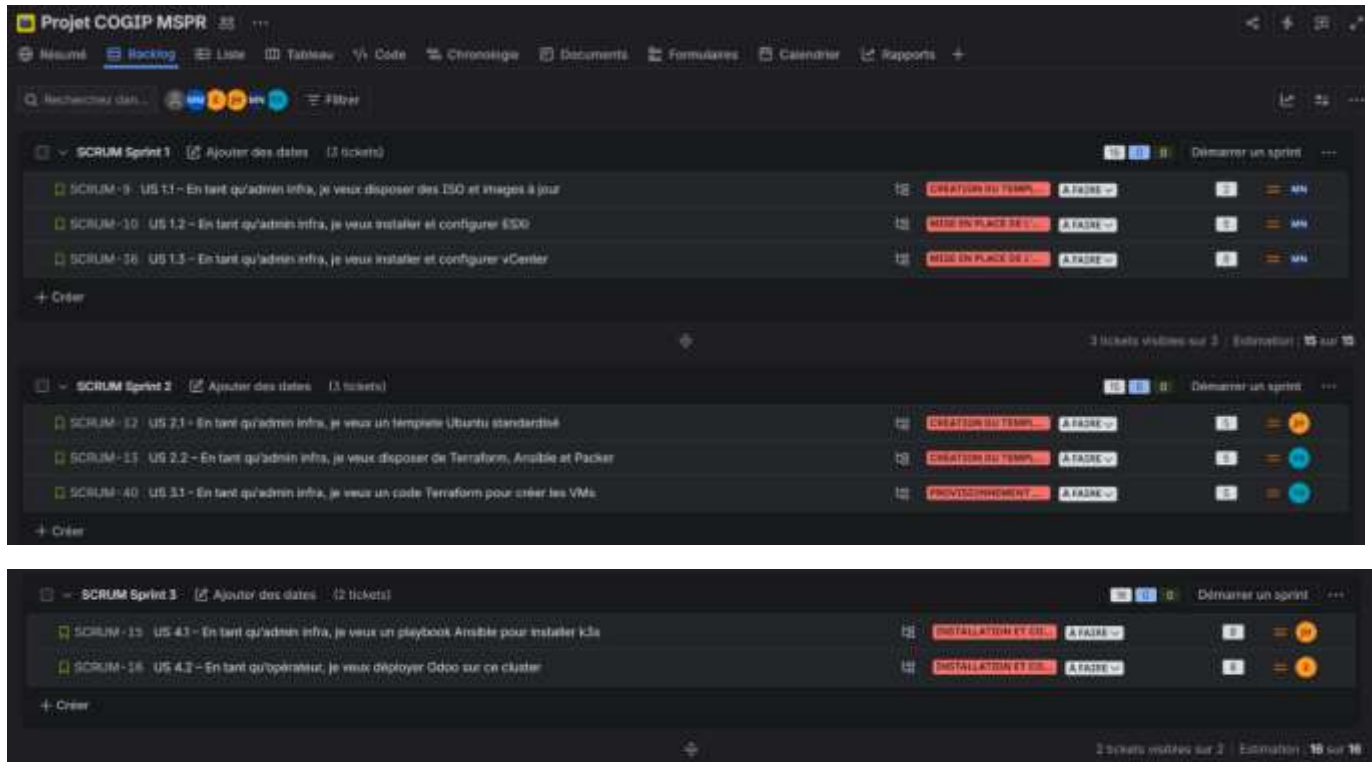


Figure 2 : Vue de notre Product Backlog, de nos Sprints chiffrés et de la répartition des User

Afin de piloter le travail de l'équipe au quotidien, nous avons configuré notre tableau Kanban en respectant strictement les étapes de validation nécessaires à un projet d'infrastructure as Code (IaC). Notre flux de travail est organisé selon les colonnes suivantes : **À faire** -> **En cours** -> **Revue Technique** -> **Terminé** :

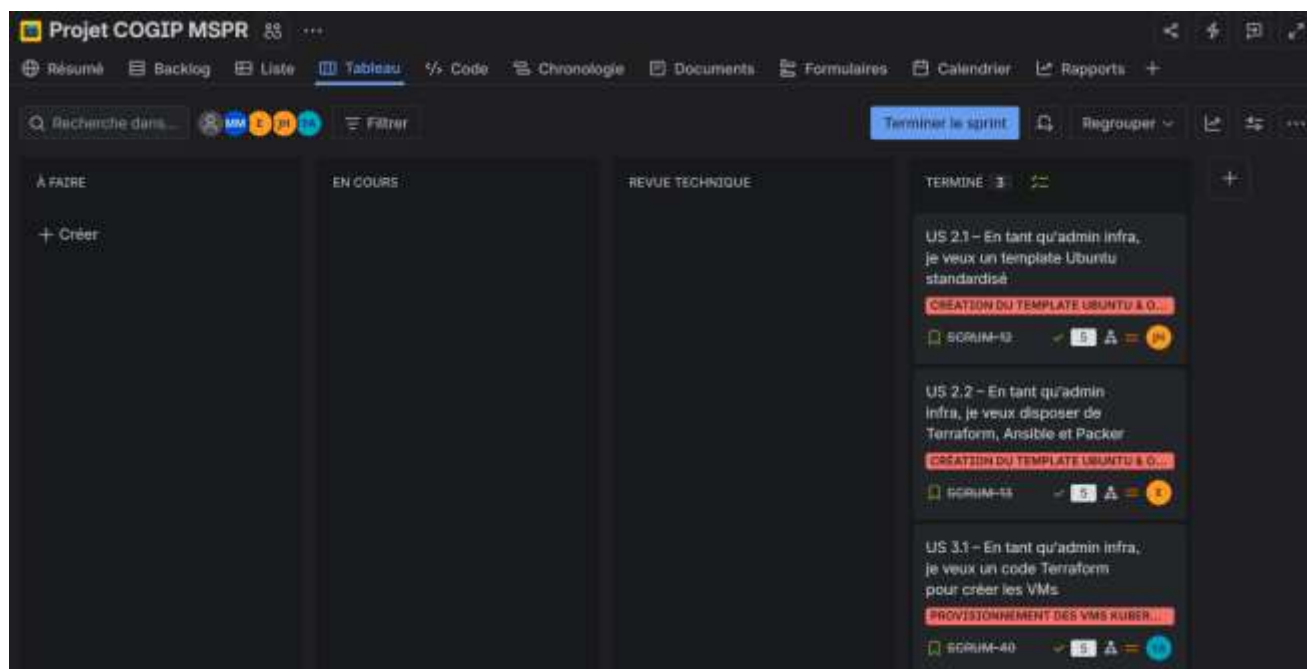


Figure 3 : Tableau Kanban en fin d'itération (Sprint 2)

Processus de Revue Technique et de Communication La colonne "Revue Technique" joue un rôle central dans notre démarche de qualité. Lorsqu'un membre de l'équipe termine l'écriture d'une recette Terraform ou d'un Playbook Ansible, le ticket est déplacé dans cette colonne. Nous organisons alors une relecture croisée de manière collective, souvent en Pair-Programming, afin de déceler d'éventuelles erreurs humaines avant le déploiement final sur le cluster.

De plus, notre projet JIRA est interconnecté avec notre outil de communication d'équipe. Chaque changement de statut d'un ticket génère une notification automatique dans notre canal dédié, ce qui fluidifie notre processus de "Check-in/Check-out" quotidien et facilite grandement la coordination en cas de télétravail.

Indicateurs de Performance et Amélioration Continue Pendant toute la durée des sprints, l'équipe a exécuté les tâches définies en se réunissant chaque jour pour un Daily Scrum de 15 minutes, favorisant ainsi la transparence et la résolution rapide des blocages. En fin de sprint, une *Sprint Review* nous permet de valider l'incrément d'infrastructure déployé et d'ajuster nos priorités. Enfin, lors des *Sprint Rétrospectives*, nous analysons collectivement nos processus afin de définir des axes d'amélioration continue.

Pour piloter notre vélocité de manière quantitative, nous nous sommes appuyés sur le Burndown Chart (voir Figure 4). Il nous a permis de visualiser l'évolution de notre effort jour après jour et de vérifier que nous tenions la cadence prévue (ligne orange) jusqu'à la clôture réussie du sprint.

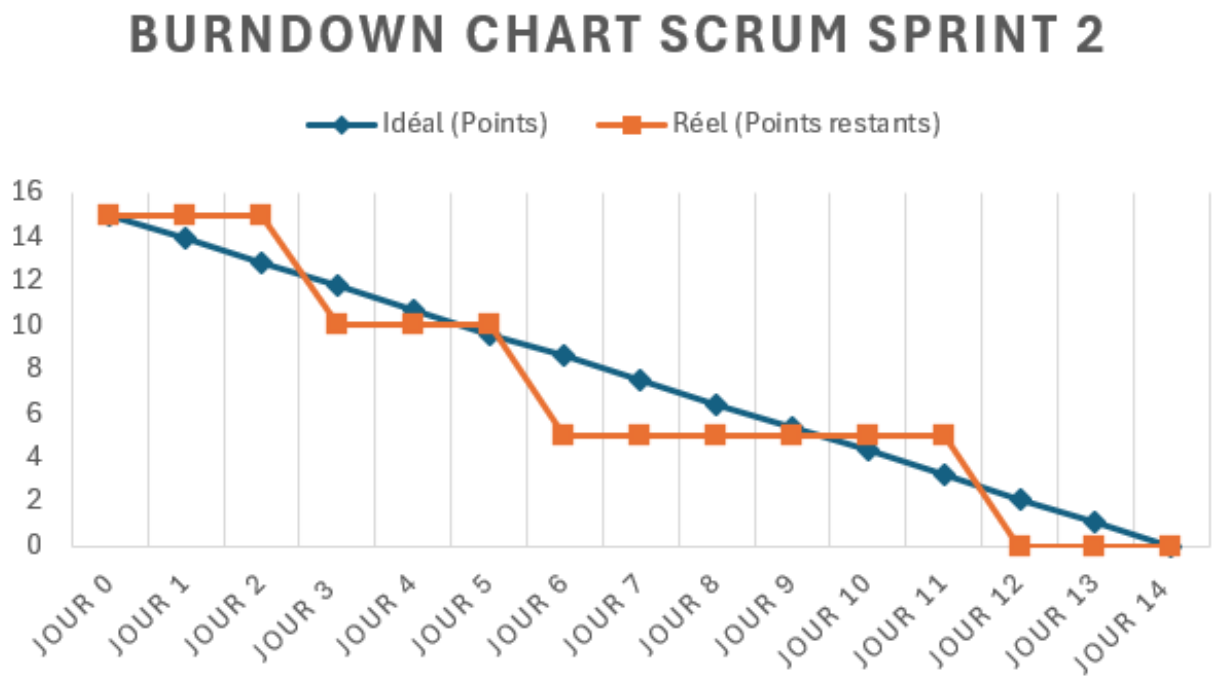


Figure 4 : Burndown Chart illustrant la vélocité et la performance de l'équipe sur le Sprint 2.

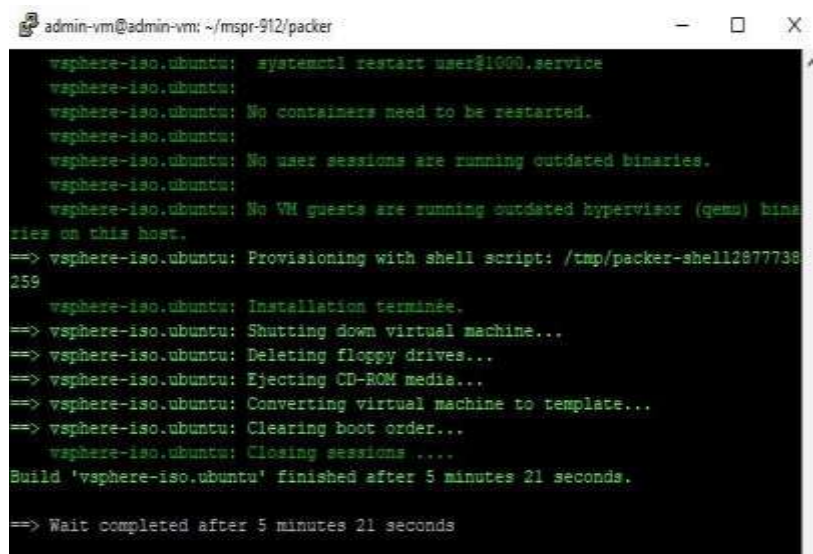
M4 : Packer

Avant de démarrer les différentes missions techniques du projet, nous avons procédé à la mise en place de l'environnement de virtualisation en installant ESXi, l'hyperviseur de VMware, sur un serveur physique, ainsi que vSphere, l'interface de gestion centralisée. Cette infrastructure constitue la base de tout le déploiement, car elle permet d'héberger et d'administrer les machines virtuelles nécessaires à la réalisation du cluster Kubernetes, ainsi qu'à l'ensemble des services du projet. Grâce à cet environnement, nous avons pu ensuite automatiser les étapes suivantes à l'aide de Packer, Terraform et Ansible, dans un cadre maîtrisé, sécurisé et adapté aux besoins de l'entreprise COGIP.

Dans ce projet, nous avons utilisé Packer pour automatiser la création d'images systèmes standardisées, servant de base au déploiement de machines virtuelles via Terraform. Cette méthode garantit une configuration homogène sur toutes les machines, tout en réduisant les erreurs humaines et le temps d'installation. Chaque machine créée à partir de cette image bénéficie des mêmes réglages, logiciels préinstallés et comptes utilisateurs.

Le fichier principal **ubuntu-template.pkr.hcl** décrit les instructions pour générer une image Ubuntu prête à l'emploi dans un environnement VMware. Il spécifie les ressources allouées, l'image ISO à utiliser, ainsi que les commandes à exécuter automatiquement, comme l'installation de paquets nécessaires au bon fonctionnement du système. Les fichiers **user-data** et **meta-data**, situés dans le dossier `http`, permettent une installation entièrement automatique du système : configuration du clavier, création d'un utilisateur avec mot de passe sécurisé, activation de SSH, réseau DHCP et installation de pilotes d'intégration.

Grâce à cette automatisation, nous sommes en mesure de fournir à l'entreprise COGIP une image système prête à l'usage, fiable, facilement réutilisable et idéale pour des déploiements en série. En cas de besoin, une nouvelle machine identique peut être générée en quelques minutes, ce qui assure une grande réactivité et une parfaite cohérence de l'infrastructure.



```
admin-vm@admin-vm: ~/mspr-912/packer
vsphere-iso.ubuntu: systemctl restart user@1000.service
vsphere-iso.ubuntu:
vsphere-iso.ubuntu: No containers need to be restarted.
vsphere-iso.ubuntu: No user sessions are running outdated binaries.
vsphere-iso.ubuntu: No VM guests are running outdated hypervisor (qemu) binaries on this host.
==> vsphere-iso.ubuntu: Provisioning with shell script: /tmp/packer-shell2877738259
vsphere-iso.ubuntu: Installation terminée.
==> vsphere-iso.ubuntu: Shutting down virtual machine...
==> vsphere-iso.ubuntu: Deleting floppy drives...
==> vsphere-iso.ubuntu: Ejecting CD-ROM media...
==> vsphere-iso.ubuntu: Converting virtual machine to template...
==> vsphere-iso.ubuntu: Clearing boot order...
vsphere-iso.ubuntu: Closing sessions ....
Build 'vsphere-iso.ubuntu' finished after 5 minutes 21 seconds.

==> Wait completed after 5 minutes 21 seconds
```

Figure 5 : Réussite mission Packer

Vous trouverez l'ensemble de ces fichiers en **annexe 3**.

M5 : Terraform

Nous avons utilisé Terraform pour automatiser la création et la configuration des machines virtuelles nécessaires au déploiement du cluster Kubernetes. Plutôt que de tout créer manuellement, Terraform nous permet, via des fichiers de configuration, de décrire précisément les ressources à mettre en place (machines, réseau, stockage, etc.), rendant l'infrastructure rapide à déployer, cohérente et facilement reproductible.

Les fichiers utilisés sont complémentaires et organisés autour de fonctions précises. Le fichier **provider.tf** établit la connexion avec le serveur VMware (vSphere), tandis que **variables.tf** et **terraform.tfvars** définissent et injectent les paramètres essentiels (réseau, identifiants, template utilisé). Le fichier **data.tf** récupère les ressources déjà existantes sur l'environnement vSphere (datacenter, hôte, réseau).

Les fichiers **vm-k3s.tf** et **vm-nfs.tf** décrivent la création automatisée des machines du cluster : un nœud principal, deux workers et un serveur de stockage. Le fichier **outputs.tf** permet d'afficher les adresses IP des machines créées, facilitant leur utilisation par la suite. Enfin, **locals.tf** permet d'ajuster rapidement des constantes comme le nombre de nœuds sans devoir modifier plusieurs fichiers.

Cette organisation permet à l'entreprise COGIP de déployer une infrastructure complète en quelques minutes, avec un contrôle total sur la configuration, et sans intervention manuelle fastidieuse.

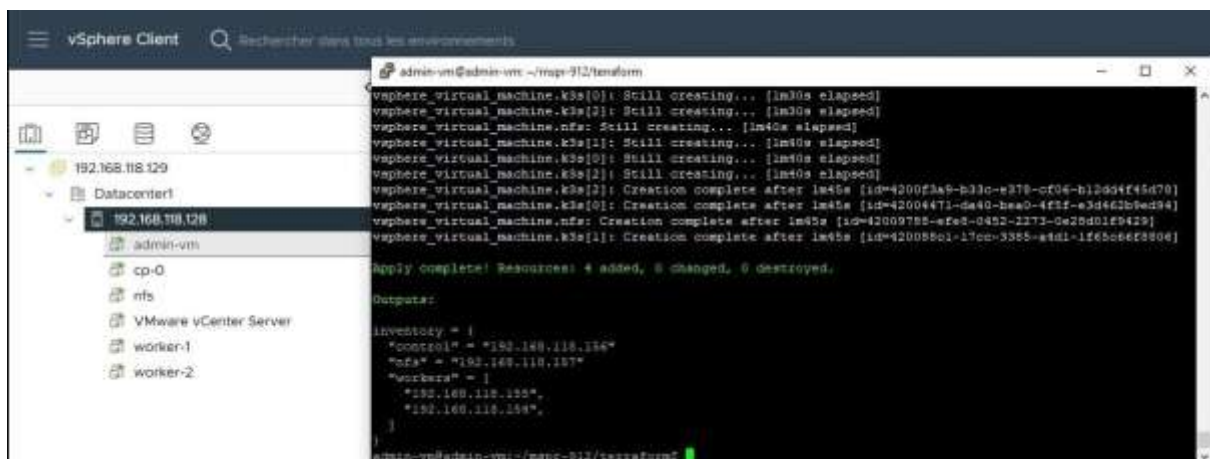


Figure 6 : Réussite mission Terraform

Vous trouverez l'ensemble de ces fichiers en **annexe 4**.

M6 : Ansible, déploiement de Kubernetes

Pour automatiser le déploiement du cluster Kubernetes sur l'infrastructure mise en place avec Terraform, nous avons utilisé Ansible, un outil d'automatisation. L'ensemble des fichiers Ansible permet de configurer chaque machine selon son rôle dans le cluster (nœud principal, workers, serveur de stockage). Le fichier **inventory.ini** répertorie toutes les machines et leurs adresses IP, tandis que **ansible.cfg** définit les réglages globaux (comme les droits administrateurs).

Le fichier principal **playbook_k3s.yml** installe automatiquement le cluster Kubernetes K3s sur les machines, en utilisant un rôle prédéfini. D'autres playbooks complémentaires permettent d'étendre l'espace disque (**playbook_extend_root.yml**) et de configurer un serveur de stockage partagé NFS (**playbook_storage.yml**). Les fichiers **control.yml** et **workers.yml** précisent certains paramètres réseau propres à chaque type de nœud. Enfin, **requirements.yml** permet d'installer tous les modules externes nécessaires à l'exécution des playbooks.

Grâce à cette architecture, l'installation du cluster est entièrement automatisée, rapide et reproductible, ce qui répond pleinement aux attentes de l'entreprise COGIP en matière de fiabilité, de gain de temps et de facilité de maintenance.

```
PLAY RECAP *****
master1 : ok=76 changed=1 unreachable=0 failed=0 s
kipped=58 rescued=0 ignored=0
afal : ok=1 changed=0 unreachable=0 failed=0 s
kipped=1 rescued=0 ignored=0
worker1 : ok=88 changed=12 unreachable=0 failed=0 s
kipped=61 rescued=0 ignored=0
worker2 : ok=58 changed=12 unreachable=0 failed=0 s
kipped=61 rescued=0 ignored=0
```

Figure 7 : Déploiement cluster

```
ubuntu@master1:~$ sudo kubectl get nodes
NAME        STATUS    ROLES                  AGE    VERSION
master1     Ready     control-plane,master   33m    v1.32.4+k3s1
worker1     Ready     <none>                 76s    v1.32.4+k3s1
worker2     Ready     <none>                 77s    v1.32.4+k3s1
```

Figure 8 : Vérification du Cluster

```
ubuntu@master1:~$ sudo kubectl exec -it demo-app -- sh -c 'echo OK >/data/check && cat /data/check'
E0525 11:11:19.497344 244097 websocket.go:296] Unknown stream id 1, discarding message
OK
ubuntu@master1:~$
```

Figure 9 : Mise en place NFS

Vous trouverez l'ensemble de ces fichiers en **annexe 5**.

M7 : Ansible, déploiement d'Odoo et mise en place de l'Ingress

Dans cette mission, nous avons utilisé Ansible pour automatiser le déploiement d'Odoo, une application de gestion d'entreprise, sur notre cluster Kubernetes. L'objectif était de rendre Odoo accessible depuis l'extérieur du cluster de manière sécurisée et également de garantir la persistance de ses données via un stockage partagé (NFS). Pour ce faire, nous avons déployé MetalLB grâce au fichier **playbook_metallb.yml**, afin d'attribuer dynamiquement une adresse IP locale aux services exposés sur le cluster.

Le déploiement d'Odoo est réalisé à l'aide du fichier **playbook_odoo.yml**, qui utilise Helm pour automatiser son installation. Ce playbook configure également un certificat TLS (via **odoo.key** et **odoo.crt**) permettant une connexion sécurisée en HTTPS. L'accès externe à l'application est assuré par un Ingress, une passerelle d'entrée configurée via le routeur Traefik, déjà intégré dans K3s. Grâce au fichier **traefik-values.yml**, nous avons activé les ports 80 et 443, tout en redirigeant automatiquement le trafic vers une version sécurisée (HTTPS). Le nom de domaine interne utilisé pour accéder à Odoo est **odoo.mspr.local**.

Enfin, nous avons validé le bon fonctionnement du stockage persistant avec le fichier **test-pvc.yml**, qui simule l'utilisation d'un volume partagé par une application. L'ensemble de cette architecture permet à l'entreprise COGIP de bénéficier d'un déploiement automatisé, fiable et sécurisé de son applicatif métier, prêt à être répliqué ou adapté à d'autres environnements.

```
admin-m@admin-mn:~/mspr-912/ansible$ ansible-playbook -i inventory.ini playbook_metallb.yml -u admin
BECOME password:

PLAY [Deploy MetalLB] *****
TASK [Gathering Facts] *****
ok: [admin-m]

TASK [Add MetalLB Helm repo] *****
ok: [admin-m]

TASK [Install MetalLB via Helm] *****
[WARNING]: The default libconfig check was fail to report changes in certain cases. Install helm diff >= 3.4.1 for better
results.
ok: [admin-m]

TASK [Ensure Kubernetes Python module is present] *****
changed: [admin-m]

TASK [Create IPAddressPool for MetalLB] *****
changed: [admin-m]

TASK [Create L2Advertisement for MetalLB] *****
changed: [admin-m]

PLAY RECAP *****
admin-m  : ok=6    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

admin-m@admin-mn:~/mspr-912/ansible$ ^C
admin-m@admin-mn:~/mspr-912/ansible$ kubectl -n kube-system get svc traefik -o
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
traefik   LoadBalancer  10.43.38.198     150.148.128.200  80:30514/TCP,443:30419/TCP  5m
```

Figure 10 : Installation MetalLB

```

admin-vn@admin-vm:~/snp-912/ansible$ ansible-playbook -i inventory.ini playbook_odoo.yml -u ubuntu
SECURE password:

PLAY [Deploy Odoo (TLS ingress + NFS persistence)] *****

TASK [Gathering Facts] *****
ok: [master1]

TASK [Install Kubernetes Python module] *****
ok: [master1]

TASK [Create odoo namespace] *****
ok: [master1]

TASK [Add Bitnami Helm repository] *****
ok: [master1]

TASK [Create TLS secret for mdm.snp.local] *****
ok: [master1]

TASK [Deploy Odoo Helm chart] *****
(WARNING): The default idempotency check can fail to report changes in certain cases. Install helm diff >= 3.4.1 for better results.
ok: [master1]

PLAY RECAP *****
master1: 1 ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

Figure 11 : Déploiement Odoo

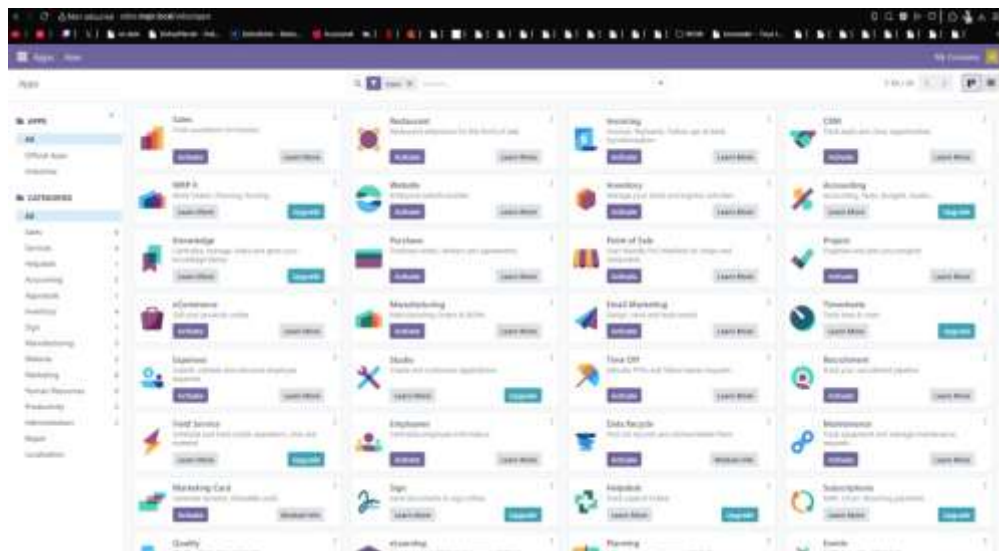


Figure 12 : Réussite mise en place Odoo

Vous trouverez l'ensemble de ces fichiers en **annexe 6**.

Conclusion

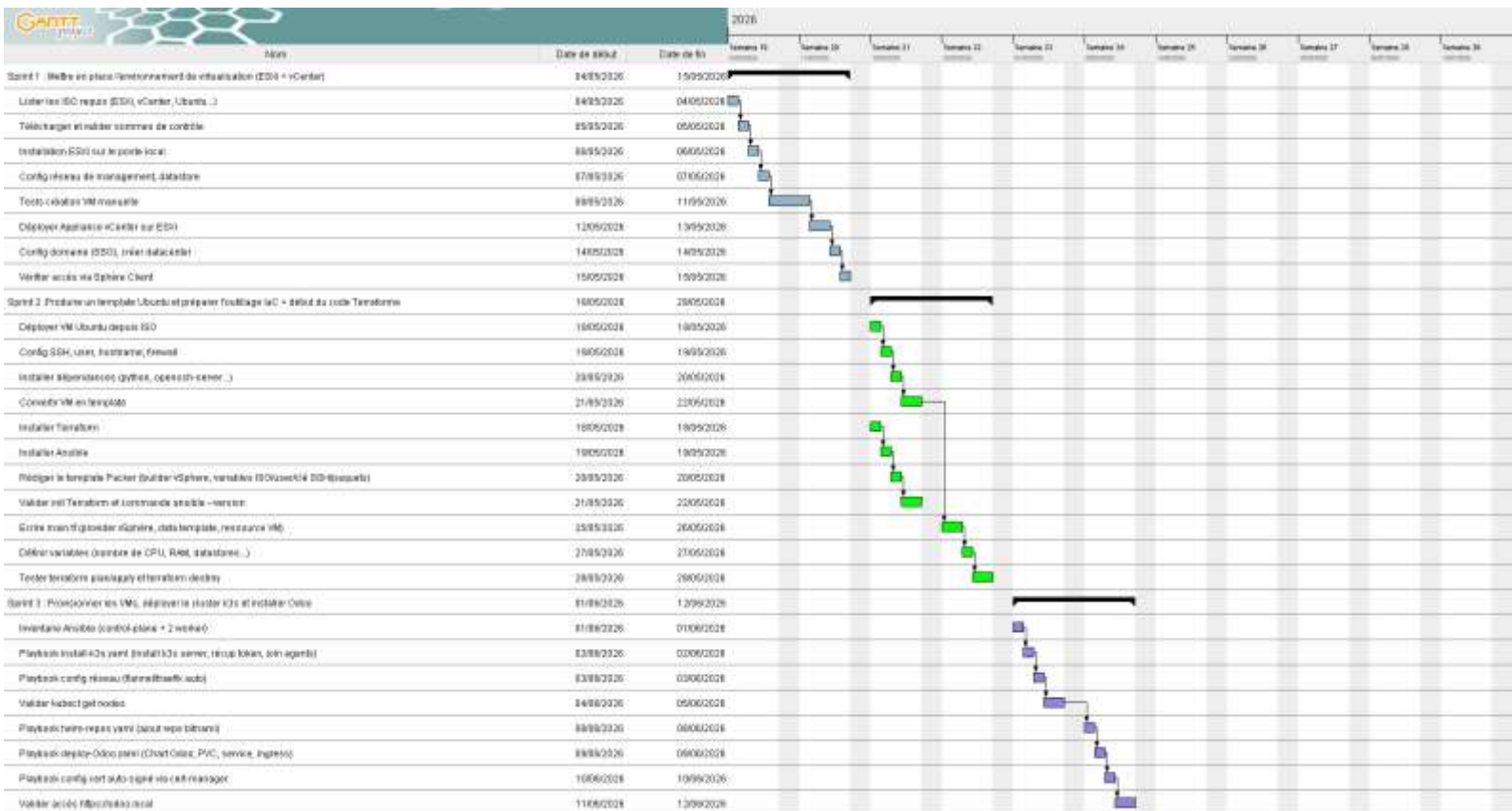
Ce projet nous a permis d'expérimenter concrètement la méthode agile, en structurant notre travail autour de sprints courts, d'Epics et d'User Stories. L'utilisation d'un tableau Kanban via JIRA, combinée aux Daily meetings et aux revues techniques collectives, a favorisé une organisation claire et dynamique. Cependant, cette approche agile, bien que bénéfique, a représenté un véritable défi, notamment dans la planification précise des tâches, la gestion du temps et la coordination entre les membres. Malgré ces difficultés, nous avons su adapter notre rythme et faire évoluer nos pratiques pour tirer pleinement parti de la méthode.

Nous avons aussi veillé à instaurer un environnement de travail inclusif, respectueux des besoins et des différences de chacun. La mise en place d'une charte éthique, l'adaptation des missions selon les capacités de chacun, et la valorisation d'une communication bienveillante ont contribué à une dynamique d'équipe saine, propice à la réussite collective.

Sur le plan technique, la mise en œuvre de l'infrastructure avec Packer, Terraform et Ansible a été particulièrement formatrice. Elle nous a permis d'aborder des concepts complexes comme le déploiement automatisé, le clustering Kubernetes et la gestion des services Ingress. Là encore, les difficultés techniques n'ont pas manqué avec les erreurs de configuration, l'incohérence réseau ou les bugs de déploiement. Mais, elles ont été l'occasion de développer notre rigueur, notre réactivité et notre autonomie. Au final, ce projet a non seulement consolidé nos compétences, mais nous a aussi fait prendre conscience de l'importance d'une gestion de projet agile structurée, résiliente et collaborative.

Annexes

Annexe 1 : Diagramme de Gantt



Annexe 2 : Environnement de travail inclusif, tolérant et multiculturel

Afin de garantir un environnement de travail inclusif, respectueux et tolérant, nous avons réfléchi à plusieurs axes d’actions visant à prendre en compte la diversité des profils au sein de l’équipe projet. L’objectif est de créer un cadre de travail où chacun peut s’épanouir et contribuer efficacement, quelles que soient ses origines culturelles, sa situation de handicap ou ses préférences personnelles.

1. Mise en place d’une charte éthique de travail

Une charte éthique de travail sera élaborée dès le début du projet. Elle établira des principes fondamentaux, tels que :

- La bienveillance dans les échanges,
- Le respect des différences culturelles, linguistiques et religieuses,
- La tolérance zéro envers les comportements discriminatoires ou excluants,
- La prise en compte des contraintes personnelles ou familiales.

Cette charte sera présentée à tous les membres de l’équipe, signée individuellement, puis rappelée régulièrement en réunion, afin de l’ancrer dans les pratiques quotidiennes.

2. Intégration des personnes en situation de handicap psycho-moteur

Nous mettrons en œuvre des mesures concrètes pour accueillir et accompagner les personnes en situation de handicap, notamment psycho-moteur :

- Adaptation de la charge de travail : attribution de tâches réalisables en télétravail, avec un rythme adapté.
- Outils d’accessibilité : mise à disposition de logiciels ergonomiques, claviers adaptés, transcription automatique des réunions, support visuel simplifié.
- Accompagnement personnalisé : un référent sera disponible pour répondre aux besoins spécifiques et effectuer les ajustements nécessaires en continu.
- Communication asynchrone privilégiée : afin de laisser à chacun le temps d’analyser et de répondre aux messages selon ses capacités et son organisation personnelle.

3. Gestion multiculturelle et prévention des malentendus

Dans un contexte où les membres peuvent provenir de cultures différentes, nous mettrons en œuvre :

Des réunions de sensibilisation aux différences culturelles et aux biais inconscients Une formation à la communication non violente et inclusive.

- L'usage d'un langage simple et neutre, accessible à tous les niveaux de maîtrise du français.
- Des supports de travail bilingues si besoin (français/anglais).

Nous favoriserons également les temps d'écoute et de partage pour permettre à chacun de faire part de ses besoins ou de ses incompréhensions, dans un cadre sécurisé.

4. Flexibilité et bien-être

Pour garantir un équilibre entre performance et bien-être, nous adopterons :

- Une politique de flexibilité horaire, permettant aux membres de gérer leur emploi du temps en fonction de leurs contraintes personnelles ou médicales.
- Des points d'équipe réguliers centrés sur le moral, la charge de travail et la satisfaction de chacun.
- Un système de feedback anonyme pour permettre de signaler tout malaise ou suggestion d'amélioration.

L'environnement de travail que nous mettrons en place est pensé pour être humain, ouvert, adaptatif et solidaire. Ces mesures concrètes visent à faire de l'inclusion une réalité vécue et non un simple objectif théorique. En valorisant la diversité et en accompagnant les vulnérabilités, nous visons à créer un cadre propice à la réussite collective et à l'épanouissement individuel.

Annexe 3 : Scripts Packer

Fichier ubuntu-template.pkr.hcl

```

#####
# Required package
#####
require("dplyr")

#####
# Variables
#####
name = "Optimal non-linear model"
version = "1.0.0"

#####
# Parameters
#####

#####
# Functions
#####

#####
# Main function
#####
main = function() {
  # Load data
  data = read.csv("data.csv")

  # Split data
  data_train = data %>% sample_n(80)
  data_test = data %>% sample_n(20)

  # Train model
  model = train(data_train, y ~ ., method = "lm")

  # Predict
  pred = predict(model, data_test)

  # Evaluate
  rmse = sqrt(mean((pred - data_test$y)^2))

  # Print results
  cat("RMSE:", rmse, "\n")
}

#####
# Run main function
#####
main()

```

Fichier user-data

```
1 #cloud-config
2 autoinstall:
3   version: 1
4   locale: fr_FR
5   keyboard:
6     layout: fr
7   identity:
8     hostname: ubuntu-template
9     username: ubuntu
10    password: "$0$rounds=4096$50qdl8qX/mq3KwJC2ndg0Kvq}k0N0M4QevE3e3oubC9AxB/c0RPabR.VP9dXSxJnu3xsE9PSU11P6BRQAdPfyxibm0ZC8FD71"
11  ssh:
12    install-verver: true
13    allow-pw: true
14  network:
15    version: 2
16    ethernets:
17      eth0:
18        dhcp: true
19        match:
20          driver: vxemv3
21        set-name: eth0
22        dhcp: true
23  storage:
24    layout:
25      name: direct
26  packages:
27    - open-vm-tools
28  late-commands:
29    - echo "ubuntu ALL=(ALL) NOPASSWD:ALL" > /target/etc/sudoers.d/ubuntu
```

Fichier meta-data

```
1 instance-id: ubuntu-template-autoinstall
2 local-hostname: ubuntu-template
3
```

Annexe 4 : Scripts Packer

Fichier provider.tf

```
1 terraform {
2   required_providers {
3     vsphere = {
4       source = "hashicorp/vsphere"
5       version = "~> 2.7"
6     }
7   }
8 }
9
10 provider "vsphere" {
11   user           = var.vsphere_user
12   password       = var.vsphere_pass
13   vsphere_server = var.vsphere_server
14   allow_unverified_ssl = true
15 }
```

Fichier variables.tf

```
1 ~ variable "vsphere_user" {
2   type = string
3 }
4
5 ~ variable "vsphere_pass" {
6   type = string
7 }
8
9 ~ variable "vsphere_server" {
10  type = string
11 }
12
13 ~ variable "network_name" {
14   type = string
15   default = "VM Network"
16 }
17
18 ~ variable "datastore" {
19   type = string
20   default = "datastore1"
21 }
22
23 ~ variable "template_name" {
24   type = string
25   default = "ubuntu-22-04-template"
26 }
27
28 ~ variable "vm_folder" {
29   type = string
30   default = "k3s-lab"
31 }
```

Fichier terraform.tfvars

```
1 # Credentials et vCenter
2 vsphere_user = "Administrator@VSPHERE.LOCAL"
3 vsphere_pass = ""
4 vsphere_server = "192.168.118.129"
```

Fichier data.tf

```
1 - data "vsphere_datacenter" "dc" {
2   name = "Datacenter1"
3 }
4
5 - data "vsphere_host" "host" {
6   name           = "192.168.118.118" # You ESXi standalone
7   datacenter_id = data.vsphere_datacenter.dc.id
8 }
9
10 - data "vsphere_datastore" "ds" {
11   name           = var.datastore
12   datacenter_id = data.vsphere_datacenter.dc.id
13 }
14
15 - data "vsphere_network" "net" {
16   name           = var.network_name
17   datacenter_id = data.vsphere_datacenter.dc.id
18 }
19
20 - data "vsphere_virtual_machine" "tpl" {
21   name           = var.template_name
22   datacenter_id = data.vsphere_datacenter.dc.id
23 }
24
25 - locals {
26   resource_pool_id = data.vsphere_host.host.resource_pool_id
27 }
```

Fichier vm-k3s.tf

```
1 - resource "vsphere_virtual_machine" "k3s" {
2   count           = local.cp_count + local.worker_count
3   name           = count.index == 0 ? "cp-0" : "worker-${count.index}"
4   resource_pool_id = local.resource_pool_id
5   datastore_id   = data.vsphere_datastore.ds.id
6   num_cpus       = 2
7   memory         = 4096
8   guest_id       = data.vsphere_virtual_machine.tpl.guest_id
9
10  network_interface {
11    network_id = data.vsphere_network.net.id
12    adapter_type = "vmxnet3"
13  }
14
15  disk {
16    label       = "disk0"
17    size        = count.index == 0 ? 20 : 30
18    thin_provisioned = true
19  }
20
21  clone {
22    template_uuid = data.vsphere_virtual_machine.tpl.id
23  }
24 }
```

Fichier vm-nfs.tf

```
1 - resource "vsphere_virtual_machine" "nfs" {
2   name           = "nfs"
3   resource_pool_id = local.resource_pool_id
4   datastore_id   = data.vsphere_datastore.ds.id
5   num_cpus       = 1
6   memory         = 1024
7   guest_id       = data.vsphere_virtual_machine.tpl.guest_id
8
9   network_interface {
10    network_id = data.vsphere_network.net.id
11    adapter_type = "vmxnet3"
12  }
13
14  disk {
15    label       = "disk0"
16    size        = 10
17    thin_provisioned = true
18  }
19
20  clone {
21    template_uuid = data.vsphere_virtual_machine.tpl.id
22  }
23 }
```

Fichier outputs.tf

```

1  output "inventory" {
2    value = {
3      nfs      = vsphere_virtual_machine.nfs.default_ip_address
4      control = vsphere_virtual_machine.k3s[0].default_ip_address
5      workers = [
6        for idx in range(1, local.worker_count + 1) :
7          vsphere_virtual_machine.k3s[idx].default_ip_address
8      ]
9    }
10 }

```

Fichier locals.tf

```

1  √ locals {
2    cp_count = 1
3    worker_count = 2
4  }

```

Annexe 5 : Scripts Ansible pour le déploiement de Kubernetes

Fichier ansible.cfg

```
1 [defaults]
2 inventory = inventory.ini
3 stdout_callback = yaml
4 retry_files_enabled = False
5 deprecation_warnings = False
6
7 [privilege_escalation]
8 become = True
9 become_method = sudo
10 become_user = root
11 become_ask_pass = True
12
13 [ssh_connection]
14 ssh_args = -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null
```

Fichier inventory.ini

```
1 [control]
2 master1 ansible_host=192.168.118.150
3
4 [workers]
5 worker1 ansible_host=192.168.118.153
6 worker2 ansible_host=192.168.118.152
7
8 [nfs]
9 nfs1 ansible_host=192.168.118.151
10
11 [k3s_cluster:children]
12 control
13 workers
14
15 [all:vars]
16 ansible_user=ubuntu
17 ansible_ssh_common_args='-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null'
18 k3s_registration_address=192.168.118.150
```

Fichier playbook_extend_root.yml

```
1 - name: Extend root partition and filesystem
2   hosts: k3s_cluster,nfs
3   become: true
4   gather_facts: false
5
6   tasks:
7     - name: Extend the partition
8       command: growpart /dev/sda 2
9       register: g
10      changed_when: "'NOCHANGE' not in g.stdout"
11
12     - name: Resize the filesystem
13       command: resize2fs /dev/sda2
14       when: g is changed
```

Fichier playbook_k3s.yml

```
1  ---
2  - name: Fix hostname
3    hosts: all
4    become: true
5    tasks:
6      - name: Set hostname to inventory alias
7        ansible.builtin.hostname:
8          name: "{{ inventory_hostname }}"
9          when: ansible_hostname != inventory_hostname
10
11 - name: Install K3s cluster
12   hosts: k3s_cluster
13   become: true
14   vars:
15     k3s_state: installed
16     k3s_become_for_all: true
17     k3s_registration_address: 192.168.118.150
18     k3s_become: true
19   roles:
20     - role: xanmanning.k3s
```

Fichier requirements.yml

```
1  ---
2  roles:
3    - name: xanmanning.k3s # rôle K3s tout-en-un
4
5  collections:
6    - name: community.general # module hostname + callback yaml
7    - name: kubernetes.core # helm_repository / helm / k8s
8    - name: community.kubernetes
```

Fichier playbook_storage.yml

```
1 ---
2 #####
3 # E Serveur NFS
4 #####
5 - name: Configure NFS server
6   hosts: nfs
7   become: true
8   tasks:
9     - name: Install NFS kernel server
10       ansible.builtin apt:
11         name: nfs-kernel-server
12         state: present
13         update_cache: true
14
15     - name: Export directory
16       ansible.builtin file:
17         path: /srv/k3s
18         state: directory
19         owner: nobody
20         group: nogroup
21         mode: '0777'
22
23     + name: Declare export
24       ansible.builtin lineinfile:
25         path: /etc/exports
26         line: "/srv/k3s 192.168.1.0/24(rw,sync,no_subtree_check,no_root_squash)"
27         create: yes
28
29     + name: Reload exports
30       ansible.builtin command: exportfs -ra
31
32 #####
33 # D Provisionneur K3s NFS
34 #####
35 - name: Install NFS external provisioner (Helm)
36   hosts: control
37   become: true
38   vars:
39     kubeconfig_path: "/etc/rancher/k3s/k3s.yaml"
40     nfs_server_ip: >-
41       [{ hostvars[groups['nfs']][0]].get('ansible_host', hostvars[groups['nfs']][0].ansible_default_ipv4.address) } ]
42   tasks:
43     - name: Ensure Helm is installed
44       ansible.builtin shell: |
45         curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
46       args:
47         creates: /usr/local/bin/helm
48
49     - name: Add Helm repo
50       kubernetes.core.helm_repository:
51         name: nfs-subdir
52         repo_url: https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner
53       kubeconfig: "{{ kubeconfig_path }}"
54
55     - name: Deploy chart
56       kubernetes.core.helm:
57         name: nfs-client
58         chart_ref: nfs-subdir/nfs-subdir-external-provisioner
59         create_namespace: nfs-provisioner
60         create_namespace: true
61       kubeconfig: "{{ kubeconfig_path }}"
62       values:
63         nfs:
64           server: "{{ nfs_server_ip }}"
65           path: /srv/k3s
66         storageClass:
67           name: nfs-client
68           defaultClass: true
69
70 #####
71 # ❶ Paquet nfs-common sur les nœuds K3s
72 #####
73 - name: Install NFS client utils on K3s nodes
74   hosts: k3s_cluster
75   become: true
76   tasks:
77     - name: Ensure nfs-common is present
78       ansible.builtin apt:
79         name: nfs-common
80         state: present
81         update_cache: true
82
```

Fichier control.yml

```
1  ---
2  # Appliqué uniquement aux nœuds du groupe [control]
3  k3s_control_node_address: "192.168.118.150"
4  k3s_registration_address: "192.168.118.150" # IP/hostname à laquelle les workers se connectent
5
6  k3s_tls_san:
7  | - "master1"
8  | - "192.168.118.150"
9
10 # Nom lisible dans `kubectl get nodes`
11 k3s_node_name: "{{ inventory_hostname }}"
```

Fichier workers.yml

```
1  ---
2  # Nom lisible dans `kubectl get nodes`
3  k3s_node_name: "{{ inventory_hostname }}"
```

Annexe 6 : Scripts Ansible pour le déploiement d'Odoo

Fichier playbook_metallb.yml

```
1 - name: Deploy Metallb
2   hosts: control
3   become: true
4   vars:
5     kubeconfig_path: "/etc/rancher/k3s/k3s.yml"
6     pool_first: 192.168.118.198
7     pool_last: 192.168.118.199
8
9   tasks:
10    - name: Add Metallb Helm repo
11      kubernetes.core.helm_repository:
12        name: metallb
13        repo_url: https://metallb.github.io/metallb
14      kubeconfig: "{{ kubeconfig_path }}"
15
16    - name: Install Metallb via helm
17      kubernetes.core.helm:
18        name: metallb
19        chart_ref: metallb/metallb
20        release_namespace: metallb-system
21        create_namespace: true
22      kubeconfig: "{{ kubeconfig_path }}"
23
24    - name: Ensure Kubernetes Python module is present
25      ansible.builtin.pip:
26        name: kubernetes
27        executable: pip3
28
29    - name: Create IPAddressPool for Metallb
30      kubernetes.core.k8s:
31        kubeconfig: "{{ kubeconfig_path }}"
32        definition:
33          apiVersion: metallb.io/v1beta1
34          kind: IPAddressPool
35          metadata:
36            name: lan-pool
37            namespace: metallb-system
38          spec:
39            addresses:
40              - "{{ pool_first }}-{{ pool_last }}"
41
42    - name: Create L2Advertisement for Metallb
43      kubernetes.core.k8s:
44        kubeconfig: "{{ kubeconfig_path }}"
45        definition:
46          apiVersion: metallb.io/v1beta1
47          kind: L2Advertisement
48          metadata:
49            name: lan
50            namespace: metallb-system
```

Fichier traefik-values.yml

```
1 - entryPoints:
2   web:
3     address: ":80"
4     http:
5       redirections:
6         entryPoint:
7           to: websecure
8           scheme: https
9
10  websecure:
11    address: ":443"
12
13  ports:
14    web:
15      expose:
16        enabled: true
17        port: 80
18    websecure:
19      expose:
20        enabled: true
21        port: 443
22    tls:
23      enabled: true
24
25  ingressRoute:
26    dashboard:
27      enabled: true
```

Fichier playbook_odoo.yml

```
1 - name: Deploy Odoo (TLS ingress + NFS persistence)
2   hosts: k8sprod
3   become: true
4   vars:
5     kubecfg_path: "/etc/kubernetes/kubelet.conf"
6     helm_namespace: helm
7
8   tasks:
9     - name: Install Kubernetes Python module
10      ansible.builtin.pip:
11        name: python-kubernetes
12        state: present
13        upgrade_strategy: yes
14
15     - name: Create odoo namespace
16      kubernetes.core.k8s:
17        kind: Namespace
18        name: {{ kubecfg_path | basename | regex_replace('.*/') }}
19        state: present
20
21     - name: Add Bitnami helm repository
22      kubernetes.core.helm_repository:
23        name: bitnami
24        repo_url: https://charts.bitnami.com/bitnami
25        kubecfg_path: {{ kubecfg_path }}
26
27     - name: Create TLS secret for odoo.svc.local
28      kubernetes.core.k8s:
29        kind: Secret
30        name: {{ kubecfg_path | basename | regex_replace('.*/') }}
31        namespace: {{ kubecfg_path | basename | regex_replace('.*/') }}
32        data:
33          {{ lookup('file', 'odoo.crt') | b64encode }}
34          {{ lookup('file', 'odoo.key') | b64encode }}
35
36     - name: Deploy odoo helm chart
37      kubernetes.core.helm:
38        name: odoo
39        chart_ref: bitnami/odoo
40        release_namespace: helm
41        create_namespace: false
42        kubecfg_path: {{ kubecfg_path }}
43        timeout: "{{ helm_timeout }}"
44        wait: true
45        reset_values: true
46        values:
47          odoo:
48            postgresql: odoo
49            postgresql_host: postgres
50            postgresql_port: 5432
51            postgresql_user: odoo
52            postgresql_password: odoo
53            postgresql_database: odoo
54            ingress:
55              enabled: true
56              ingress_class_name: traefik
57              backend: odoo.svc.local
58              tls: true
59              extra_args:
60                - --secret=odoo-tls
61            secretName: odoo-tls
```

Fichier test-pvc.yml

```
1 ---
2 apiVersion: v1
3 kind: PersistentVolumeClaim
4 metadata:
5   name: demo-pvc
6
7 spec:
8   storageClassName: nfs-client
9   accessModes:
10     - ReadWriteOnce
11   resources:
12     requests:
13       storage: 1Gi
14
15 ---
16 apiVersion: v1
17 kind: Pod
18 metadata:
19   name: demo-app
20
21 spec:
22   containers:
23     - name: demo-app
24       persistentVolumeClaim:
25         claimName: demo-pvc
26       image:
27         name: busybox
28         imagePullPolicy: Always
29         command: ["sh", "-c", "sleep 3600"]
30       volumeMounts:
31         - mountPath: /data
32           name: demo-pvc
```

Fichier odoo.key

```
1 -----BEGIN PRIVATE KEY-----
2 MIIEVATBADAQBgkqhkiG9w0BAQEEFAASCBKYYaggSIAgEAAoIBAQCzRFOAh1SkuVz
3 TP8gSQN3bFunt1/ewdQ82rCrufQDeSV5d8LVA6nD9g1AXI535g4dPh6BqVL3wohNq
4 VMAkuu6kyqJavpQ8Y7KV+Gf+OSXG7muuTpb5d32y5Usv073BPvB8K9hm1sRcGT
5 6r0rX0HgCDD8pvSE1qPGzELhQKVzeACEptHsTFyZvygSk1YBG4Y21pv752qs11
6 XOL/V4jF3uG6lwsp#BakIo11eDyoK7VFSJw034eD7vt1006N6kb17mZoS4ml7FZc
7 xFZ/O1VipE3x9Ysy/QK6PdJJKW28j/8PQvYSIaQ1dnhwn003YMKb0UJoQEfxjnh
8 vHq5vK/VAgWBAACggEAC87yFeT0F3LM6nqRMQ1r52atQCY1heDqzC001MXIUu3s
9 oZ1RTbenER/AnhS2sMeJ09yMHTK3H245dH8sLC/yu9HnAqLoC9dMTVJkPDWaptq
10 8cMpzXvfa5Aum0PQs0ICJwReHjk1H+1AMdTG1Fh1bNZdTLwd1FzIExeHk6Yw8Z1
11 1/BrWokM/petAkeIUdIsjjLNF+YV2nlsr0LB4XX8DK1jBEy3LLnTos8j1/y8C5Gw
12 DJ8dp3J2gPLEt+nriPpQkXqJHIWqSPHhYcQQCS5oethaSuB9KUYNa80+DGA3I
13 Hqya51Q1z2kEGjMPN31uEG6K261X9v7LzLhXA25swK8gQDxH71r1cAy1peh0ct1
14 s6Gf3CgcqpfYcCwA0rTdpdJkATHda7VAW7DN07p+rj1988ta0h9jffEL881Hh
15 RV7A5Ery3aEPcDaUcFX256H6Gz8C1pz3yPkQFQK8w+17X8yITWwCQe/KMAAD
16 8TYXVEgnC4+csRiDCHo1Fr90wK8gQD0Bq80jnvposCq8t70KBPAXIXPMMAHk
17 8s/d6x1uCi8YuzpTakAr88GfQcQ1DPSa0+st4Vjk9E88W5C3Q4w51905vK8a68
18 7Lj7t08jTveNgrQhd4XqEVBqsTurs4U21FH0vq1G+q0FSPY35AeEkeSS1g6Ek1Ic
19 /A1cAB0RtwK8gDnYLEstb6CjrwkafT9Rn1akUHQ+01Ta6YHGLaDeq0c1xReuao
20 nmH1uawNkw/Stx9b59gm43PP5rC8SHnQupXvGIFPduCib03DF3PCwY1cd390E0
21 rvhac1/0EWt9y0Lc0yZzcEDM4eCVH6aay3XUKTaEsPocFYdMC855Hy1AaGABNQM
22 oXb0E5uH+em99L2TNeUqFg5k9VLjKcFUB1nh50Rku1kE73E0eYVluEk3y0F3cj
23 V0h+2rZ6uJt1b1Vak1Cw8FvdphXAPvhaENoE33F3oy0931e3lly61gFubgPk3d
24 7vnyYpsu0YyaYS2U0150W4Vx1VLD1f4U7AWA7WcGv815YU726Rb4AS05Rn1jv20
25 juW01q1rFYR1cM4Q13B1j180r/1Bb5xR2sUBFnRfc4A4a1gTn91/eGrFuPWHQ
26 1v9wBQj0FA1Xu5b82XzkfPtKu/Uynhzb8q81umtyLN/1D19abzWtIyMf1YV11otW
27 In2v0nnZh8pK7Yf4uRtj5Q==
28 -----END PRIVATE KEY-----
```

Fichier odoo.crt

```
1 -----BEGIN CERTIFICATE-----
2 MIIDFTCCAF2gAwIBAgIUe1e6vK034P4VctxqcwhY4qE9K8gwDQY7EoZIHvCNAQE1
3 BQAwGjEYMBYGA1UEAwPb2Rvby5tc3B5LmV2ZS5MB4XDTE1MDUyMTExdjc0MjE0K
4 DTI2MDUyMTExdjc0MjE0GjEYMBYGA1UEAwPb2Rvby5tc3B5LmV2ZS5MB4XDTE1MDU
5 Y2E1MDUyMTExdjc0MjE0GjEYMBYGA1UEAQMBAQEEFAASCBKYYaggSIAgEAAoIBAQCz
6 RFOAh1SkuVzTP8gSQN3bFunt1/ewdQ82rCrufQDeSV5d8LVA6nD9g1AXI535g4dPh6B
7 qVL3wohNqVMAkuu6kyqJavpQ8Y7KV+Gf+OSXG7muuTpb5d32y5Usv073BPvB8K9hm1s
8 RcGT6r0rX0HgCDD8pvSE1qPGzELhQKVzeACEptHsTFyZvygSk1YBG4Y21pv752qs11
9 XOL/V4jF3uG6lwsp#BakIo11eDyoK7VFSJw034eD7vt1006N6kb17mZoS4ml7FZcx
10 FZ/O1VipE3x9Ysy/QK6PdJJKW28j/8PQvYSIaQ1dnhwn003YMKb0UJoQEfxjnhvHq5v
11 K/VAgWBAACggEAC87yFeT0F3LM6nqRMQ1r52atQCY1heDqzC001MXIUu3soZ1RTben
12 ER/AnhS2sMeJ09yMHTK3H245dH8sLC/yu9HnAqLoC9dMTVJkPDWaptq8cMpzXvfa5A
13 um0PQs0ICJwReHjk1H+1AMdTG1Fh1bNZdTLwd1FzIExeHk6Yw8Z11/BrWokM/petA
14 keIUdIsjjLNF+YV2nlsr0LB4XX8DK1jBEy3LLnTos8j1/y8C5GwDJ8dp3J2gPLEt+nri
15 PpQkXqJHIWqSPHhYcQQCS5oethaSuB9KUYNa80+DGA3IHqya51Q1z2kEGjMPN31uEG
16 6K261X9v7LzLhXA25swK8gQDxH71r1cAy1peh0ct1s6Gf3CgcqpfYcCwA0rTdpdJk
17 ATHda7VAW7DN07p+rj1988ta0h9jffEL881HhRV7A5Ery3aEPcDaUcFX256H6Gz8C1
18 pz3yPkQFQK8w+17X8yITWwCQe/KMAAD8TYXVEgnC4+csRiDCHo1Fr90wK8gQD0Bq80j
19 nvposCq8t70KBPAXIXPMMAHk8s/d6x1uCi8YuzpTakAr88GfQcQ1DPSa0+st4Vjk9E
20 88W5C3Q4w51905vK8a687Lj7t08jTveNgrQhd4XqEVBqsTurs4U21FH0vq1G+q0FSPY
21 35AeEkeSS1g6Ek1Ic/A1cAB0RtwK8gDnYLEstb6CjrwkafT9Rn1akUHQ+01Ta6YHGL
22 aDeq0c1xReuao nmH1uawNkw/Stx9b59gm43PP5rC8SHnQupXvGIFPduCib03DF3PCwY
23 1cd390E0rvhac1/0EWt9y0Lc0yZzcEDM4eCVH6aay3XUKTaEsPocFYdMC855Hy1AaG
24 ABNQM oXb0E5uH+em99L2TNeUqFg5k9VLjKcFUB1nh50Rku1kE73E0eYVluEk3y0F3cj
25 V0h+2rZ6uJt1b1Vak1Cw8FvdphXAPvhaENoE33F3oy0931e3lly61gFubgPk3d7vny
26 Ypsu0YyaYS2U0150W4Vx1VLD1f4U7AWA7WcGv815YU726Rb4AS05Rn1jv20juW01q1
27 rFYR1cM4Q13B1j180r/1Bb5xR2sUBFnRfc4A4a1gTn91/eGrFuPWHQ1v9wBQj0FA1Xu
28 5b82XzkfPtKu/Uynhzb8q81umtyLN/1D19abzWtIyMf1YV11otWIn2v0nnZh8pK7Yf4
29 uRtj5Q==
30 -----END CERTIFICATE-----
```